

history_handler.js - Complete Documentation

File Location: `src/js/history_handler.js`

Purpose: Configuration-based system for displaying history details (complaints, feedback, requests, items)

Dependencies: `modall.js`

Used by: `admin_dashboard.js`, `student_dashboard.js`, all history modals

Overview

The `history_handler.js` file uses a clever configuration-based pattern to handle multiple history types without code duplication. Instead of writing separate functions for each history type, it defines a configuration object where each type specifies its own data, rendering functions, and HTML structure.

Core Configuration Object

history_types Configuration

```
const history_types = {
  eventRequest: { ... },
  complaint: { ... },
  feedback: { ... },
  reportedItem: { ... }
};
```

Purpose: Central configuration defining all history types in the system.

History Type: eventRequest

Config Key: `'eventRequest'`

Purpose: Track student-submitted event requests (e.g., "Can we have a coding competition?")

Configuration Structure:

```

eventRequest: {
    title: 'Event Request',
    url: '../..php/student/get_my_event_requests.php',
    searchKey: 'event_name',
    detailUrl: '../..php/admin/get_event_request.php',

    // Render list row
    renderRow: function(item) {
        return `
            <div class="history-row">
                <div class="history-title">${item.event_name}</div>
                <div class="history-date">${item.request_date}</div>
                <div class="history-status status-${item.status}">
                    ${item.status.toUpperCase()}
                </div>
            </div>
        `;
    },

    // Render detail view
    renderDetail: function(item) {
        return `
            <div class="detail-section">
                <h3>${item.event_name}</h3>
                <p><strong>Requested by:</strong> ${item.student_name}</p>
                <p><strong>Date:</strong> ${item.request_date}</p>
                <p><strong>Status:</strong> <span class="status-${item.status}">${item.status}</span></p>
                <p><strong>Description:</strong></p>
                <p>${item.description}</p>
                <p><strong>Expected Date:</strong> ${item.expected_date}</p>
                <p><strong>Admin Response:</strong></p>
                <p>${item.admin_response || 'Pending...'}</p>
            </div>
        `;
    }
}

```

Data Fields:

- event_name - Requested event name
- student_name - Student who requested
- request_date - When requested
- status - pending, approved, rejected
- description - Event description
- expected_date - When student wants it
- admin_response - Admin's reply

Status Colors:

- ☐ pending - Yellow
- ☐ approved - Green
- ☐ rejected - Red

History Type: complaint

Config Key: 'complaint'

Purpose: Track student-submitted complaints (e.g., "Event was cancelled last minute")

Configuration Structure:

```
complaint: {
  title: 'Complaint',
  url: '../..//php/student/get_my_complaints.php',
  searchKey: 'event_name',
  detailUrl: '../..//php/admin/get_complaint_details.php',

  renderRow: function(item) {
    const hasReply = item.admin_reply && item.admin_reply.length > 0;
    return `
      <div class="history-row ${hasReply ? 'has-reply' : ''}">
        <div class="history-title">${item.event_name}</div>
        <div class="history-date">${item.complaint_date}</div>
        <div class="history-status">
          ${hasReply ? '☐ Replied' : '☐ Pending'}
        </div>
      </div>
    `;
  },

  renderDetail: function(item) {
    return `
      <div class="detail-section">
        <h3>Complaint: ${item.event_name}</h3>
        <p><strong>Date Submitted:</strong> ${item.complaint_date}</p>
        <p><strong>Category:</strong> ${item.category}</p>
        <p><strong>Your Complaint:</strong></p>
        <p>${item.complaint_text}</p>

        <div class="admin-reply-section">
          <h4>Admin Reply:</h4>
          <p>${item.admin_reply || '<em>No reply yet</em>'}</p>
          <p><small>Replied: ${item.reply_date || 'N/A'}</small></p>
        </div>
      </div>
    `;
  }
}
```

Data Fields:

- event_name - Event being complained about
- complaint_date - When submitted
- category - Complaint category
- complaint_text - Full complaint message
- admin_reply - Admin's response
- reply_date - When admin replied

History Type: feedback

Config Key: 'feedback'

Purpose: Track event feedback and ratings submitted by students

Configuration Structure:

```
feedback: {
  title: 'Feedback',
  url: '../..//php/student/get_my_feedback.php',
  searchKey: 'event_name',
  detailUrl: '../..//php/admin/get_event_feedback.php',

  renderRow: function(item) {
    const stars = '□'.repeat(parseInt(item.rating) || 0);
    return `
      <div class="history-row">
        <div class="history-title">${item.event_name}</div>
        <div class="history-rating">${stars}</div>
        <div class="history-date">${item.feedback_date}</div>
      </div>
    `;
  },

  renderDetail: function(item) {
    const stars = '□'.repeat(parseInt(item.rating) || 0);
    return `
      <div class="detail-section">
        <h3>Feedback: ${item.event_name}</h3>
        <p><strong>Rating:</strong> ${stars} (${item.rating}/5)</p>
        <p><strong>Date:</strong> ${item.feedback_date}</p>
        <p><strong>Your Feedback:</strong></p>
        <p>${item.feedback_text}</p>
        <p><strong>Suggestions for Next Time:</strong></p>
        <p>${item.suggestions || 'None'}</p>
      </div>
    `;
  }
}
```

Data Fields:

- event_name - Event being reviewed
- rating - 1-5 star rating
- feedback_text - Written feedback
- suggestions - Suggestions for improvement
- feedback_date - When submitted

Rating Display:

- 1 star: □
 - 2 stars: □□
 - 3 stars: □□□
 - 4 stars: □□□□
 - 5 stars: □□□□□
-

History Type: reportedItem

Config Key: 'reportedItem'

Purpose: Track lost/found items reported by students

Configuration Structure:

```
reportedItem: {
  title: 'Reported Item',
  url: '../..//php/student/get_reported_items.php',
  searchKey: 'item_name',
  detailUrl: '../..//php/admin/get_reported_item_detail.php',

  renderRow: function(item) {
    const icon = item.item_type === 'lost' ? '❏' : '❑';
    return `
      <div class="history-row item-${item.item_type}">
        <div class="history-title">${icon} ${item.item_name}</div>
        <div class="history-info">${item.item_type.toUpperCase()}</div>
        <div class="history-date">${item.report_date}</div>
      </div>
    `;
  },

  renderDetail: function(item) {
    const icon = item.item_type === 'lost' ? '❏ Lost' : '❑ Found';
    const statusBadge = item.status === 'resolved'
      ? '<span class="badge-resolved">✓ Resolved</span>'
      : '<span class="badge-pending">❏ Pending</span>';

    return `
      <div class="detail-section">
        <h3>${icon}: ${item.item_name}</h3>
        <p><strong>Status:</strong> ${statusBadge}</p>
        <p><strong>Location:</strong> ${item.location}</p>
        <p><strong>Date Reported:</strong> ${item.report_date}</p>
        <p><strong>Description:</strong></p>
        <p>${item.description}</p>
        <p><strong>Contact:</strong> ${item.contact_info}</p>
        <p>${item.found_comment ? `
          <div class="found-comment">
            <h4>Admin Note:</h4>
            <p>${item.found_comment}</p>
          </div>
        ` : ''}</p>
      </div>
    `;
  },
}
```

Data Fields:

- `item_name` - Name of item

- `item_type` - 'lost' or 'found'
 - `location` - Where lost/found
 - `report_date` - When reported
 - `description` - Item description
 - `contact_info` - Student contact
 - `photo` - Item photo path
 - `status` - pending, resolved
 - `found_comment` - Admin note if found
-

Core Functions

openHistoryModal(type)

Purpose: Opens generic history modal for specified type.

Parameters:

- `type` (string) - History type ('eventRequest', 'complaint', 'feedback', 'reportedItem')

Returns: void

What it does:

1. Gets config for history type
2. Opens history modal HTML
3. Calls `loadHistory(type)` to populate

Code:

```
function openHistoryModal(type) {  
  if (!history_types[type]) {  
    console.error(`Unknown history type: ${type}`);  
    return;  
  }  
  
  const config = history_types[type];  
  
  openModal('../frontends/history_modal.html', 'historyModal', function() {  
    // Set title  
    const title = document.getElementById('historyModalTitle');  
    if (title) title.textContent = config.title;  
  
    // Load data  
    loadHistory(type);  
  });  
}
```

loadHistory(type)

Purpose: Fetch and display history list for specified type.

Parameters:

- `type` (string) - History type

Returns: void (async)

What it does:

1. Gets config for type
2. Fetches data from configured URL
3. Renders rows using config.renderRow()
4. Adds click handlers to open details

Code:

```
async function loadHistory(type) {
  const config = history_types[type];
  const container = document.getElementById('historyList');

  if (!config || !container) return;

  try {
    const res = await fetch(config.url);
    const items = await res.json();

    if (!Array.isArray(items) || items.length === 0) {
      container.innerHTML = '<p>No history items.</p>';
      return;
    }

    container.innerHTML = items.map(item => `
      <div onclick="openHistoryDetail('${type}', ${item.id || item.request_id || item.complaint_id})">
        ${config.renderRow(item)}
      </div>
    `).join('');

  } catch (err) {
    console.error(err);
    container.innerHTML = '<p class="error">Failed to load history.</p>';
  }
}
```

openHistoryDetail(type, id)

Purpose: Open detail view for specific history item.

Parameters:

- type (string) - History type
- id (integer) - Item ID

Returns: void

What it does:

1. Gets config for type
2. Fetches detail data from configured detailUrl
3. Renders using config.renderDetail()
4. Shows in detail panel

Code:

```

async function openHistoryDetail(type, id) {
  const config = history_types[type];
  const detailDiv = document.getElementById('historyDetail');

  if (!config || !detailDiv) return;

  try {
    const res = await fetch(`${config.detailUrl}?id=${id}`);
    const item = await res.json();

    detailDiv.innerHTML = config.renderDetail(item);
    detailDiv.style.display = 'block';

  } catch (err) {
    console.error(err);
    detailDiv.innerHTML = '<p class="error">Failed to load details.</p>';
  }
}

```

searchHistory(type, searchTerm)

Purpose: Search history items by term.

Parameters:

- `type` (string) - History type to search
- `searchTerm` (string) - Search query

Returns: void

What it does:

1. Gets config for type
2. Identifies search field from config.searchKey
3. Filters items matching search
4. Re-renders filtered results

Code:

```

function searchHistory(type, searchTerm) {
  const config = history_types[type];
  const searchKey = config.searchKey;

  // Get all items (would need to load fresh or cache)
  const filtered = currentHistoryItems.filter(item =>
    String(item[searchKey] || '').toLowerCase().includes(searchTerm.toLowerCase())
  );

  const container = document.getElementById('historyList');
  container.innerHTML = filtered.map(item => `
    <div onclick="openHistoryDetail('${type}', ${item.id})">
      ${config.renderRow(item)}
    </div>
  `).join('');
}

```

Modal Structure

history_modal.html Expected Structure

```
<div class="modal-content">
  <div class="modal-header">
    <h2 id="historyModalTitle">History</h2>
    <input type="text" id="historySearch" placeholder="Search..."
      onkeyup="handleHistorySearch(this.value)">
    <span class="close" onclick="closeModal('historyModal')">X</span>
  </div>

  <div class="modal-body">
    <div id="historyList" class="history-list">
      <!-- History rows rendered here -->
    </div>
    <div id="historyDetail" class="history-detail">
      <!-- Detail view rendered here -->
    </div>
  </div>
</div>
```

Two-Panel Layout

Left Panel: List View

Event Request
2024-05-15
Status: PENDING
Complaint
2024-05-14
Status: ☐ REPLIED
Feedback
☐ ☐ ☐ ☐ ☐
2024-05-10

Right Panel: Detail View

```
| Complaint: Event A |
|                   |
| Submitted: 2024-05-14 |
| Category: Quality  |
|                   |
| Your Message:      |
| The venue was too... |
|                   |
| Admin Reply:        |
| Thank you, we will.. |
```

Configuration-Based Benefits

□ Advantages of This Pattern

1. DRY (Don't Repeat Yourself)

```
// Instead of this repeated code for each type:
function loadComplaints() { /* fetch */ }
function loadFeedback() { /* fetch */ }
function loadRequests() { /* fetch */ }

// We have this single function:
function loadHistory(type) { /* handles all */ }
```

2. Easy to Add New Types

```
// Just add new config object:
history_types.newType = {
  title: 'New Type',
  url: '../..//php/get_new_type.php',
  searchKey: 'name',
  renderRow: function(item) { /* ... */ },
  renderDetail: function(item) { /* ... */ }
};
```

3. Centralized Configuration All history types in one place, easy to audit and modify.

4. Consistent Behavior All history types work the same way, better UX.

Data Flow Example

Complete Workflow: Viewing Complaint History

```
Student opens "My Complaints"
  ↓
openHistoryModal('complaint')
  ↓
Gets config: history_types['complaint']
  ↓
Opens history_modal.html
  ↓
loadHistory('complaint') called
  ↓
Fetches get_my_complaints.php
  ↓
Response: Array of complaints
  ↓
For each complaint, renders using
  config.renderRow(item)
  ↓
List shows:
  - Complaint title
  - Date submitted
  - Status (Pending/Replied)
  ↓
User clicks a complaint
  ↓
openHistoryDetail('complaint', id)
  ↓
Fetches get_complaint_details.php?id=5
  ↓
Response: Full complaint data
  ↓
Renders using config.renderDetail(item)
  ↓
Detail panel shows:
  - Full complaint text
  - Admin reply
  - Reply date
```

Performance Considerations

□ Current Strengths

- Single modal reused for all types
- Efficient rendering with `.map().join('')`
- Configuration-based reduces code duplication

⚠ Potential Improvements

1. Pagination

```
const loadHistory = (async function(type, page = 1) {  
    const config = history_types[type];  
    const limit = 10;  
    const offset = (page - 1) * limit;  
    // Add pagination to URL  
});
```

2. Caching

```
const historyCache = new Map();  
  
async function loadHistory(type) {  
    if (historyCache.has(type)) {  
        renderHistoryList(historyCache.get(type), type);  
        return;  
    }  
    // Fetch and cache...  
}
```

3. Split Detail and List Modals

```
openHistoryListModal(type); // Lightweight modal  
openHistoryDetailModal(type, id); // Full detail modal
```

Summary

history_handler.js provides:

Feature	Purpose
history_types	Central config for all history types
openHistoryModal()	Open history modal for type
loadHistory()	Fetch and display list
openHistoryDetail()	Show item details
searchHistory()	Filter results by term

Creates a flexible, configuration-based system for displaying any type of history without code duplication.

Last Updated: May 22, 2026

File Size: ~250 lines of code

Complexity: Medium (pattern-based)

Criticality: High (used by admin + student)

Design Pattern: Configuration Object + Strategy Pattern